Diagnóstico del Aprendizaje y Parada Temprana

Prevención de Sobreajuste, Checkpointing Óptimo y Evaluación de Predicciones

---

Laboratorio de Inteligencia Artificial — PyTorch Demanda

12 de octubre de 2026

Deep Learning & Modelado Predictivo (`red_neuronal.py`)

El Problema: ¿Por qué Entrenar a Ciegas es Peligroso?

El Enfoque Tradicional Fijo

Fijar un número arbitrario de épocas (ej. `EPOCAS = 100`) plantea un dilema irresoluble a priori:

- **Muy pocas épocas**: El modelo se queda en **subajuste** (*underfitting*), sin extraer patrones no lineales.
- **Demasiadas épocas**: La red memoriza el ruido muestral en **sobreajuste** (*overfitting*).

La Solución Natural

Transformar el criterio de parada en un **mecanismo dinámico**:

$$t_{\text{stop}} = \text{mín}\,\{t \mid \Delta t_{\text{sin mejora}} \geq P\}$$

donde P es la **paciencia** del optimizador.

Riesgos de Usar el Último Modelo

- En la época final $E = 100$, los pesos θ_{100} rara vez son los mejores.
- En problemas reales, la pérdida de validación oscila y con frecuencia **empeora al final**.
- **Conclusión clave**: Necesitamos guardar el mejor punto histórico θ^* (*checkpointing*) y cortar el cómputo cuando ya no hay ganancia real.

Fundamento Teórico: Regularización Implícita

Equivalencia con Regularización L_2 (Weight Decay)

Existe una equivalencia matemática formal entre detener el descenso de gradiente temprano y aplicar regularización de Tikhonov (L_2):

$$\theta_t \approx \theta^* - (I - \eta H)^t(\theta^* - \theta_0)$$

- El número de pasos t actúa como el recíproco del hiperparámetro de penalización:

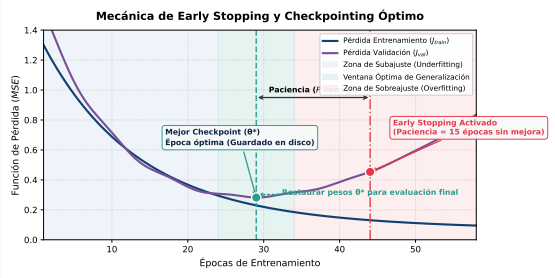
$$t \sim \frac{1}{\lambda_{L_2}}$$

- Detener el entrenamiento temprano restringe la norma euclídea del vector de pesos $\|\theta\|_2$, impidiendo que crezca sin límite para ajustar ruido.

Tres Fases en la Trayectoria de Pérdida

1. **Fase I: Aprendizaje de señal rápida**: J_{train} y J_{val} descienden conjuntamente a gran velocidad.
2. **Fase II: Cuenca de generalización**: J_{val} alcanza su mínimo global en θ^*.
3. **Fase III: Memorización espuria**: $J_{\text{train}} \to 0$ mientras J_{val} diverge debido a la alta varianza del espacio latente.

Mecánica de Parada Temprana y Checkpointing Óptimo



1. Checkpoint Óptimo (θ^*)

- Se guarda en disco en la época de menor J_{val}.
- Garantiza la máxima generalización empírica.

2. Ventana de Paciencia (P)

- Margen de tolerancia (ej. 15 épocas).
- Evita detenerse ante ruido térmico del mini-batch.

3. Restauración de Pesos

- Tras la parada, se descarta θ_{stop}.
- Se carga θ^* para evaluar en datos de prueba.

Implementación en `red_neuronal.py`

1. Monitoreo y Guardado de Checkpoint

En cada época t:

1. Calcular J_{val} sin gradientes (`torch.no_grad()`).
2. Si $J_{\text{val}} < J_{\text{val}}^{\text{mín}} - \delta$:
   - $J_{\text{val}}^{\text{mín}} \leftarrow J_{\text{val}}$
   - $t^* \leftarrow t$
   - `torch.save(red.state_dict(), "mejor_modelo.pth")`
   - Reiniciar contador: `epocas_sin_mejora = 0`
3. En caso contrario:
   - `epocas_sin_mejora += 1`

2. Condición de Parada y Carga Óptima

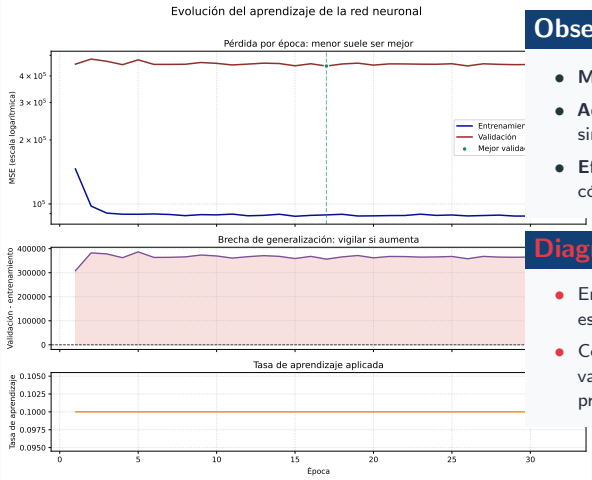
Condición de parada anticipada:

- Si `epocas_sin_mejora >= PACIENCIA`:
  - Imprimir diagnóstico de detención.
  - Ejecutar `break` para salir del bucle.

Carga del mejor checkpoint antes del test:

- `red.load_state_dict(torch.load("mejor_modelo.pth"))`
- `red.eval()`
- Predicción sobre `X_prueba_tensor`.

Curva de Aprendizaje Real con Early Stopping



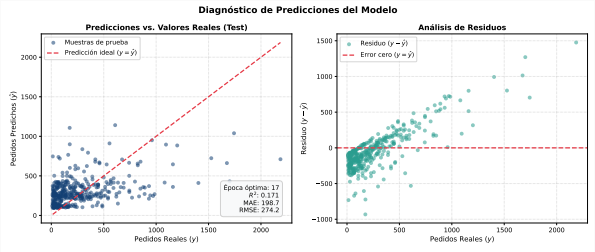
Observación Experimental

- **Mejor Época Registrada**: **Época 17** ($MSE_{\text{val}} = 445,235$).
- **Activación de Parada**: **Época 32** tras 15 épocas consecutivas sin mejora (paciencia = 15).
- **Eficiencia y Ahorro**: Se evitaron 68 épocas redundantes de cómputo.

Diagnóstico de la Brecha (ΔJ)

- En el panel central, la brecha de generalización (Val − Train) se estabilizó alrededor de la época 17.
- Continuar el descenso no reducía la cota superior del error de validación, confirmando que la red alcanzó su capacidad predictiva útil.

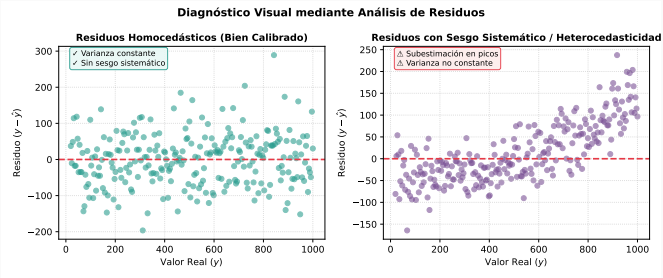
Diagnóstico Visual 1: Predicciones vs. Valores Reales



¿Qué nos revela esta gráfica?

- **Línea de 45° ($y = \hat{y}$)**: Representa la predicción perfecta. Si el modelo fuera exacto, todos los puntos descansarían sobre la recta discontinua roja.
- **Dispersión en zona habitual ($< 1,000$ pedidos)**: La red captura la tendencia central con error acotado ($MAE \approx 198$).
- **Saturación en demandas extremas ($> 1,500$)**: Los pedidos inusualmente altos sufren subestimación. La red predice conservadoramente alrededor de 500–1, 000.

Diagnóstico Visual 2: Análisis de Residuos ($e_i = y_i - \hat{y}_i$)



Comportamiento Ideal (Homocedasticidad)

- Los residuos forman una banda simétrica centrada en 0.
- La varianza del error es constante independientemente de la magnitud de y.
- Garantiza que no hay información sistemática desperdiciada.

Comportamiento en Demanda Real (Heterocedasticidad)

- En datasets de comida rápida, los picos masivos generan residuos positivos gigantes ($y \gg \hat{y}$).
- **Pauta de mejora**: Aplicar transformación logarítmica a la salida ($\log(1 + y)$) o función de pérdida Huber/Smooth L1.

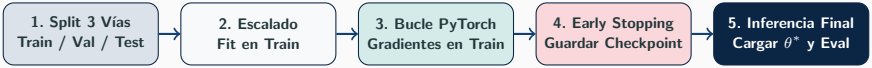
Métricas de Desempeño: ¿Cómo se complementan?

Métrica	Fórmula	Sensibilidad	Interpretación Práctica
MAE (Error Absoluto)	$\frac{1}{N}\sum \lvert y_i - \hat{y}_i \rvert$	Lineal; robusto a valores atípicos	Magnitud intuitiva: $\ll$*En promedio nos equivocamos en* $\pm$198 *pedidos*$\gg$.
RMSE (Raíz de MSE)	$\sqrt{\frac{1}{N}\sum (y_i - \hat{y}_i)^2}$	Cuadrática; penaliza severamente errores grandes	Castiga desvíos en días pico: útil para gestión logística y rotura de stock.
Coeficiente R^2	$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$	Normalizado respecto a la varianza total	Porcentaje de variabilidad explicado (1.0 = perfecto, 0 = promedio).

Lección del Diagnóstico

Un R^2 modesto en datos brutos ocurre por la enorme asimetría en pedidos extremos (13 a 12, 137). El análisis visual de residuos explica de inmediato lo que un único número tabular no puede revelar.

Protocolo de Ingeniería en Aprendizaje Supervisado



Buenas Prácticas de Paciencia

- Si el batch size B es pequeño (más ruido de gradiente), **aumentar la paciencia** ($P \geq 20$).
- Si usamos un optimizador adaptativo con decay (ej. scheduler), sincronizar la paciencia con la reducción de tasa.

Integridad Metodológica

- **Cero fuga de información**: El test set jamás interviene en la decisión de detenerse ni en el escalado.
- **Determinismo**: La semilla fija (seed 42) garantiza experimentos rigurosamente reproducibles.

Conclusiones y Siguientes Pasos

Lo que Hemos Consolidado

1. **Control de sobreajuste**: Early stopping detiene el cómputo en el punto de óptima generalización empírica.
2. **Checkpointing persistente**: El archivo `mejor_modelo.pth` conserva el estado óptimo θ^* listo para despliegue.
3. **Diagnóstico integral**: La doble gráfica (predicciones vs reales y análisis de residuos) permite entender dónde y por qué falla el modelo.

La Continuación Natural Próxima

- **Transformación de la variable objetivo**: Aplicar $\log(1 + y)$ para comprimir la varianza extrema de los pedidos.
- **Regularización adicional**: Probar `Dropout(0.2)` o penalización L_2 (`weight_decay`).
- **Módulo de Inferencia en Producción**: Crear una función `predecir_pedido(precio, promo, semana)` para responder consultas operativas.

¿Preguntas o Discusión?

Código fuente: `red_neuronal.py` **y** `graficar_aprendizaje.py`
Pesos óptimos guardados: `mejor_modelo.pth`